Laboratorio de Triage de Defectos

Criterios para analizar defectos, clasificar impacto, hallar causas raíz y proponer quality gates

Sesión guía para estudiantes

D-101

Pago aprobado sin recibo

Severidad ? Prioridad ?

D-103

Login intermitente en horas pico

Severidad ? Prioridad ?

D-105

PDF tarda más de 35 s

Severidad ? Prioridad ?

OBJETIVOS DE LA SESIÓN

Qué deben aprender

01

Triage

Diferenciar severidad y prioridad, acordar responsables, fechas objetivo y criterios de cierre.

02

Causa raíz

Aplicar 5 Whys e Ishikawa para pasar del síntoma a causas verificables y no a opiniones.

03

Prevención

Traducir hallazgos en plan de mejora, KPIs y reglas automáticas dentro del pipeline CI/CD.

MÉTODO DE TRABAJO

Flujo del laboratorio

1 Entrada

Defectos reportados con **evidencia mínima**: pasos, logs, stack traces, capturas y severidad propuesta.

2 Triage

Se acuerdan **severidad, prioridad, responsable, due date** y decisión sobre hotfix, release o backlog.

3 Análisis

Cada defecto priorizado pasa por **5 Whys e Ishikawa** para distinguir causas validadas y supuestos pendientes.

4 Salida

Plan de mejora con acciones priorizadas, KPI de verificación y quality gates que reduzcan recurrencia.

EL CASO

Defectos reportados del caso

ID	Defecto	Impacto principal	Evidencia inicial
D-101	**Pago aprobado sin recibo**	Afecta operación crítica y confianza del usuario	Captura + timeout del servicio
D-102	**Descuento calculado incorrectamente**	Error de negocio y cobro equivocado	Datos de prueba + comparación esperada
D-103	**Login intermitente en horas pico**	Indisponibilidad parcial del sistema	HTTP 500 + logs + métrica de carga
D-104	**Botón Enviar fuera de pantalla**	Problema funcional en móvil	Captura responsive
D-105	**PDF tarda más de 35 s**	Degradación severa de rendimiento	Log de consulta lenta + tiempo medido

Imágenes ilustrativas

Evidencia de ejemplo por defecto



D-101

Pago aprobado sin recibo

Captura + timeout del servicio



D-103

Login intermitente en horas pico

HTTP 500 + logs + métrica de carga



D-104

Botón Enviar fuera de pantalla

Captura responsive



D-105

PDF tarda más de 35 s

Log de consulta lenta + tiempo medido

CLASIFICAR · PASO 1

Qué revisar en cada defecto

Reproducibilidad

¿Ocurre siempre, a veces o bajo ciertas condiciones?

Impacto

¿Bloquea una transacción crítica, afecta datos o genera pérdidas?

Alcance

¿Afecta a uno, varios o todos los usuarios y ambientes?

Urgencia

¿Puede esperar o compromete la salida a producción?

Workaround

Si existe alternativa temporal, la prioridad puede ajustarse.

Evidencia

La decisión debe sustentarse en datos, no en intuiciones.

CLASIFICAR · PASO 2

Severidad vs prioridad

S Severidad

Mide el impacto técnico o funcional del defecto sobre el sistema.

PREGUNTA CLAVE
¿Qué tan grave es el daño?

P Prioridad

Mide la urgencia de corrección según riesgo de negocio, release y usuarios afectados.

PREGUNTA CLAVE
¿Qué tan rápido debe atenderse?

EJEMPLOS

> Un **botón mal alineado** puede tener severidad media pero prioridad alta si impide finalizar una inscripción masiva.

> Un **error raro pero destructivo** puede tener severidad crítica y prioridad alta aunque aparezca pocas veces.

CLASIFICAR · PASO 3

Escala sugerida de severidad

Crítica	Caída del sistema, corrupción de datos, pérdida económica, incumplimiento de seguridad o bloqueo de operación crítica.
Alta	Función principal afectada sin colapso total; hay impacto fuerte en negocio o usuario.
Media	Afecta funcionalidad secundaria o existe workaround razonable.
Baja	Cosmético, menor o sin impacto sustancial en operación o datos.

CLASIFICAR · PASO 4

Escala sugerida de prioridad

MAYOR URGENCIA

MENOR URGENCIA

P1

Inmediato

Debe atenderse de inmediato, bloquea release, operación o usuario crítico.

P2

Ciclo actual

Debe resolverse pronto en el ciclo actual, aunque exista contención temporal.

P3

Próximo sprint

Se agenda en backlog o próximo sprint; no compromete salida inmediata.

P4

Baja urgencia

Mejora deseable o ajuste de baja urgencia.

CLASIFICAR · PASO 5

Matriz de impacto y urgencia

SEVERIDAD	PRIORIDAD BAJA	PRIORIDAD ALTA
ALTA	**Alta severidad + baja prioridad** Raro, pero posible si no afecta release inmediata. Se vigila y planifica con fecha corta.	**Alta severidad + alta prioridad** Corregir primero. Normalmente hotfix, rollback, mitigación inmediata o bloqueo de despliegue.
BAJA	**Baja severidad + baja prioridad** Se documenta y va a backlog si no genera riesgo relevante.	**Baja severidad + alta prioridad** No destruye el sistema, pero afecta una campaña, una fecha académica o una tarea clave del usuario.

CLASIFICAR · PASO 6

Preguntas para clasificar bien

Sobre el negocio

- ¿Se pierde dinero, tiempo o confianza?
- ¿Hay una fecha crítica de matrícula o pago?
- ¿Se compromete una obligación institucional?

Sobre lo técnico

- ¿Se dañan datos o integraciones?
- ¿El sistema cae o responde lentamente?
- ¿Se puede reproducir con facilidad?

CLASIFICAR · PASO 7

Cómo decidir en la sesión de triage

1

Revisar evidencia

Sin pasos claros ni datos, la clasificación será débil.

2

Discutir impacto

Se clasifica primero el daño, luego la urgencia.

3

Acordar decisión

Corregir ahora, posponer, investigar o descartar.

4

Asignar responsable

Cada defecto debe salir con dueño claro.

5

Definir fecha

Sin compromiso temporal no hay seguimiento real.

6

Criterio de cierre

La corrección debe ser verificable con prueba y evidencia.

CAUSA RAÍZ · 5 WHYS

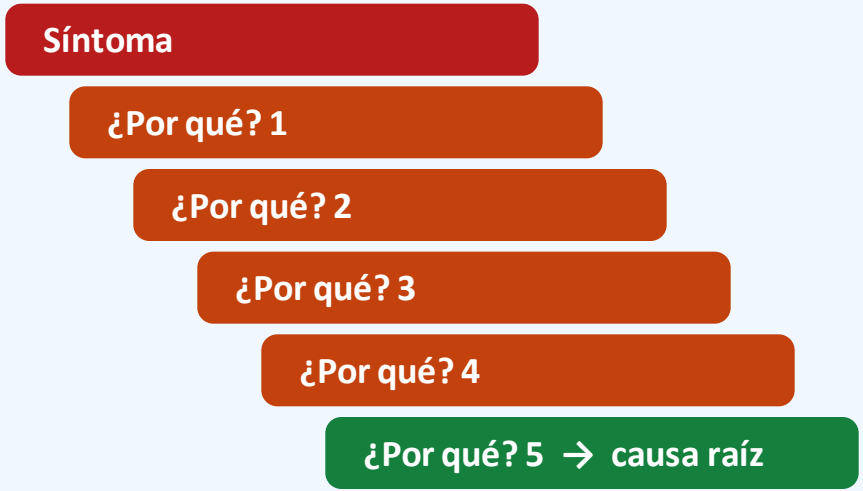
5 Whys: de síntoma a causa

OBJETIVO

Profundizar progresivamente hasta encontrar una causa plausible y verificable, en lugar de quedarse en el síntoma.

REGLA

Cada respuesta debe ser concreta, basada en evidencia y conectada lógicamente con la anterior.



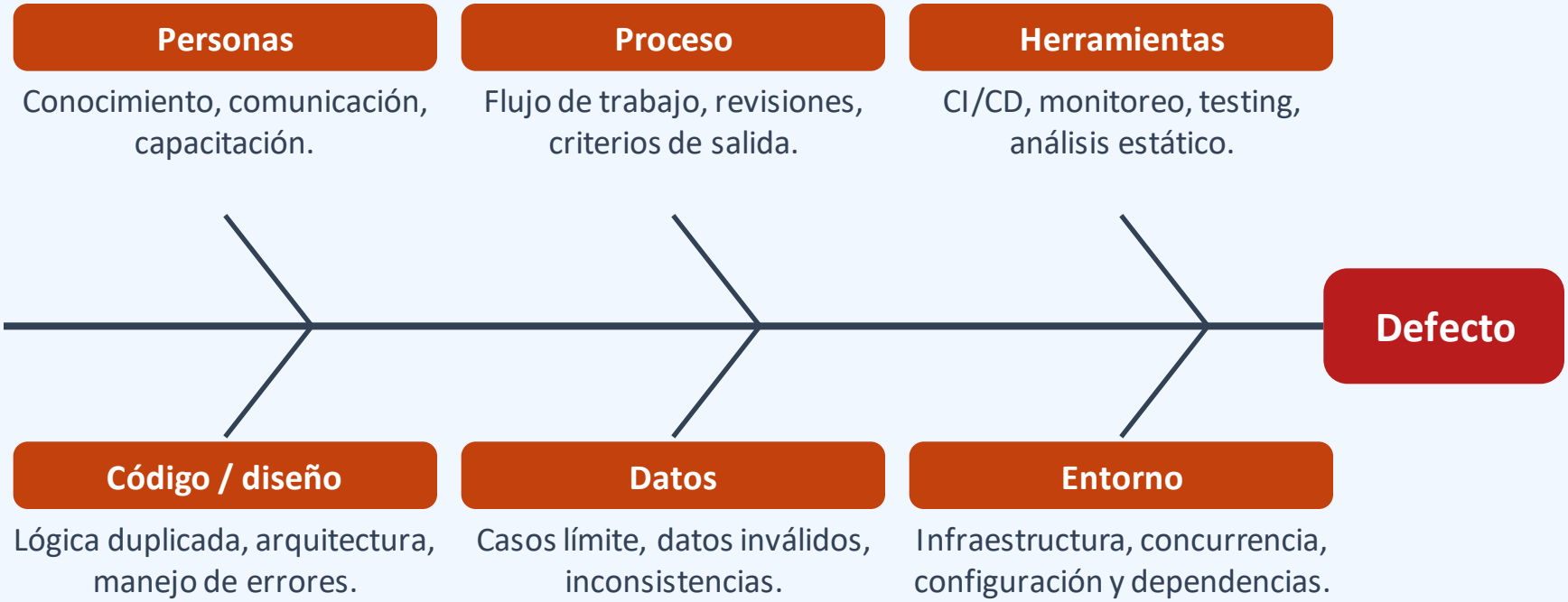
Error común: terminar en “porque el desarrollador se equivocó”. La causa útil apunta a proceso, diseño, validación, pruebas, datos, monitoreo o arquitectura.

Ejemplo ilustrativo

5 Whys aplicado a D-103

Síntoma	El login falla con HTTP 500 en horas pico.
¿Por qué? 1	Se agota el pool de conexiones a la base de datos.
¿Por qué? 2	Los intentos que terminan en timeout no liberan su conexión.
¿Por qué? 3	El manejo de errores no cierra la conexión en la ruta de excepción.
¿Por qué? 4	Las pruebas de carga no simulan fallos con alta concurrencia.
¿Por qué? 5	**Causa raíz:** El pipeline no exige prueba de carga antes del release.

Ishikawa: categorías recomendadas



MEJORA · PLAN DE ACCIÓN

Plan de mejora: qué debe contener

Acción

Debe indicar si previene, corrige, detecta o automatiza.

Responsable

Persona o rol que ejecuta y responde por el avance.

Esfuerzo

Bajo, medio o alto; útil para priorizar junto con impacto.

KPI

Métrica observable con umbral claro para verificar mejora.

EJEMPLO ILUSTRATIVO PARA EL DEFECTO D-103

Agregar prueba de carga al pipeline	**QA lead**	**Medio**	**Latencia P95 login ≤ 300 ms**

MEJORA · PRIORIZACIÓN

Priorización impacto / esfuerzo

IMPACTO

ALTO

BAJO

Alto impacto + bajo esfuerzo

Primero. Son las ganancias rápidas.

Alto impacto + alto esfuerzo

Importantes, pero requieren planeación y patrocinio.

Bajo impacto + bajo esfuerzo

Útiles si sobran recursos o mejoran orden operativo.

Bajo impacto + alto esfuerzo

Normalmente se posponen o descartan.

BAJO ESFUERZO

ALTO ESFUERZO

MEJORA · VERIFICACIÓN

KPIs útiles para verificar mejora

Error rate

Porcentaje de solicitudes fallidas en login, pagos o APIs críticas.

≤ 1%

Meta sugerida de error rate

Latencia P95

Mide rendimiento percibido en escenarios reales de carga.

≤ 300 ms

Meta sugerida P95 login

Cobertura de smoke

Valida que los flujos críticos estén protegidos por automatización.

≥ 80%

Cobertura mínima en nuevo código

MEJORA · QUALITY GATES

Quality gates que deberían proponer

Código y calidad

Coverage en nuevo código	≥ 80%
Issues críticos o bloqueantes	0
Duplicación en nuevo código	≤ 3%

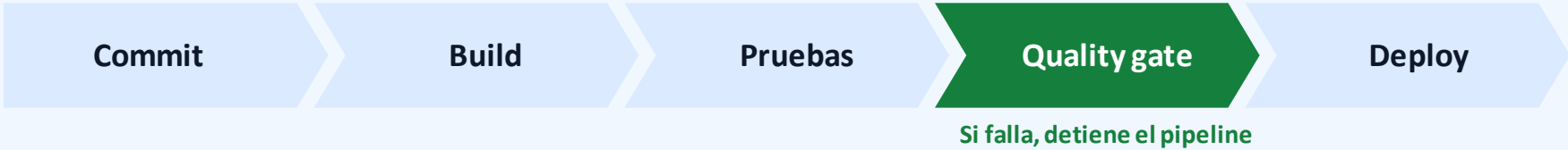
Flujos críticos

Smoke tests aprobados	100%
Error rate	≤ 1%
Latencia P95 dentro del umbral definido	≤ umbral

El criterio central: el gate **no debe ser decorativo**. Debe poder bloquear el avance del pipeline cuando el riesgo es alto.

CI/CD · EJEMPLO

Ejemplo de control en pipeline



Riesgo	Regla	Herramienta	Política
Defectos lógicos nuevos	Coverage nuevo ≥ 80%	SonarQube + pruebas unitarias	**Fail**
Fallo en login o pago	Smoke tests críticos 100%	Cypress / Playwright	**Fail-fast**
Rendimiento degradado	P95 login ≤ 300 ms	k6 / JMeter	**Warning o fail según ambiente**
Errores críticos de código	0 blocker / critical issues	SonarQube	**Fail**

CIERRE · LO QUE HAY QUE EVITAR

Errores frecuentes

1

Confundir severidad con prioridad

No son sinónimos; una evalúa daño y la otra urgencia.

2

No sustentar con evidencia

Una clasificación sin logs ni casos reproducibles pierde fuerza.

3

Proponer acciones vagas

“Mejorar pruebas” no basta; se requiere acción específica y medible.

CIERRE

Meta de la sesión

Que cada equipo pueda **justificar sus decisiones**, conectar defectos con causas y convertir el análisis en controles verificables.

Triage + Causa raíz + Mejora continua + CI/CD